

Chapter 2: Open Source Tools



Teaching Objectives:

Discover the ecosystem of Free and Open Source Software (FOSS).



summary

Chapter 2: Open Source Tools

2.1 Introduction

2.2 Development environment

2.3 Office software (LibreOffice Suite)

2.4 Collaboration (Storage and sharing)

2.5 Contributing to an open source project



Chapter 2: Open Source Tools

2.1 Introduction:

2.1.1 Definition:

Open source refers to software whose source code is publicly available, allowing users to study, edit, enhance, and distribute it freely. Unlike commercial (proprietary) software, open source promotes transparency, freedom, and collaboration.

Open source is not just a license. It is a philosophy and development model based on knowledge sharing, innovation, and collective improvement.



Chapter 2: Open Source Tools

2.1.2 Key Characteristics:

Characteristic

Free access

Source code access

Community support

Transparency

License-based freedom

Meaning

Anyone can download and use the software

Code is visible and modifiable

Developers worldwide contribute

No hidden processes or restrictions

Users rights protected by open licenses



Chapter 2: Open Source Tools

2.1.3 Historical Overview:

1960s–1970s: Academic and research institutions shared source code freely.

1983: *Richard Stallman* launched the **GNU Project**, promoting software freedom.

1985: Creation of the **Free Software Foundation (FSF)**.

1991: *Linus Torvalds* released the first version of **Linux**, accelerating global adoption.

1998: The term *Open Source* was officially coined and formalized.

Today: Open source powers modern technology including servers, mobile systems, cloud computing, and AI platforms.



Chapter 2: Open Source Tools

2.1.4 Advantages of Open Source

Cost-effective : Free to use and modify

Transparency : Source code accessible for review

Innovation : Supported by global developer communities

Security : Bugs discovered and fixed quickly

Flexibility : Customizable for specific needs

Independence : No vendor lock-in



Chapter 2: Open Source Tools

2.1.5 Disadvantages of Open Source

Requires technical skills : Installation and maintenance knowledge needed

Possible limited official support : Community-based assistance

Compatibility issues : May not always integrate with proprietary systems

Fragmentation : Multiple versions of the same tool



Chapter 2: Open Source Tools

2.1.6 Open Source Licenses

Licenses define user rights for using and modifying software.

	Type	Permissions	Example
GPL (General Public License)	Copyleft	Must remain open if modified	Linux, GNU tools
MIT License	Permissive	Allows commercial reuse	Node.js, React
Apache 2.0	Permissive + Patent protection	Encourages enterprise use with clear terms.	Hadoop, Kubernetes
BSD License	Minimal restrictions	Allows redistribution with minimal conditions.	FreeBSD, OpenSSH



Chapter 2: Open Source Tools

2.2.1 Introduction to Linux:

Linux is an open-source operating system based on the Unix model. It was created by Linus Torvalds in 1991 and has become one of the most widely used operating systems in the world.

Linux is used in:

- Servers (about 90% of web servers run on Linux)

- Supercomputers

- Cloud platforms (AWS, Azure, Google Cloud)

- Android phones (Android = Linux-based)

- Cybersecurity & networking

- Embedded systems (cars, drones, IoT)

Linux is considered the heart of open-source technology.



Chapter 2: Open Source Tools

Why is Linux important in education and industry?

Free & open

Students can install and experiment without cost

Stable & secure

Rarely crashes, resistant to malware

Industry standard

Used by companies like Google, Meta, NASA

Ideal for programming

Comes with built-in tools for developers

Excellent for networking

CLI tools for servers and networks

Teaches system fundamentals

Students learn how an OS works internally



Chapter 2: Open Source Tools

Linux Distribution

The Linux kernel is used to create complete operating systems called distributions (distros).

Ubuntu

Education, beginners, desktops

Debian

Stability, servers

Fedora

Developers, latest features

CentOS / Rocky Linux

Enterprise servers

Kali Linux

Cybersecurity and penetration testing



Chapter 2: Open Source Tools

Structure of a Linux System

Linux has a clear and organized architecture:

1. The Kernel

The core of the system.

Manages CPU, memory, hardware, and processes.

2. Shell

A program that interprets commands typed by the user.

Shell types:

Bash (default on many distros)

Zsh

Fish



Chapter 2: Open Source Tools

3. File System Hierarchy

Everything in Linux is a file, even hardware.

Important directories:

Directory	Meaning
/home	Users' personal files
/bin	Essential programs
/etc	Configuration files
/var	Logs & variable data
/root	Superuser directory
/usr	System-wide programs



Chapter 2: Open Source Tools

Basic Linux Commands

Command	Description
ls	List files
cd folder	Change directory
mkdir folder	Create a directory
rm file	Delete a file
cp file1 file2	Copy files
sudo apt update	Update package list
sudo apt install	Install software
pwd	Show current directory



Chapter 2: Open Source Tools

Practical Uses of Linux in Real Life

In universities

- Hosting Moodle platforms

- Running labs for programming courses

- Supporting student projects in Python/Java/AI

In companies

- Web servers (Apache, Nginx)

- Databases (MySQL, PostgreSQL)

- DevOps pipelines (Docker, Kubernetes)

- Cybersecurity operations

- Cloud automation



Chapter 2: Open Source Tools

For students

Learning scripting

Running web apps locally

Practicing penetration testing

Preparing for IT certifications (Linux+, LPIC, RHCSA)



Chapter 2: Open Source Tools

2.2.2 Introduction to Open Source Code Editors

A code editor is a software tool used to write, edit, and manage source code. Open-source editors offer great flexibility, extensions, and community support.

Why code editors matter in programming?

- Syntax coloring

- Auto-completion and suggestions

- Debugging tools

- Terminal integration

- Extensions for every language

- Git version control support

They help students write cleaner, faster, and more efficient code.



Chapter 2: Open Source Tools

Popular Open Source Code Editors

1. Visual Studio Code (VS Code)

The most used editor in the world
Free, open-source (MIT license)
Extensions for Python, Java, C, Web, AI, etc.
Integrated terminal
Git and GitHub integration
Intelligent suggestions

2. Atom Editor

Fully open-source
Highly customizable
Suitable for web development
Created by GitHub



Chapter 2: Open Source Tools

3. Geany / Gedit

Lightweight graphical editors

Ideal for low-end computers

Good for writing small programs (C, Python, Bash)

4. Vim (Terminal Editor)

Very fast

Runs inside the terminal

Used by professionals, sysadmins, DevOps

Steep learning curve but highly powerful



Chapter 2: Open Source Tools

2.3 LibreOffice Suite (Open-Source Office Suite)

LibreOffice is a powerful, free, and fully open-source office suite developed by The Document Foundation (TDF).

It serves as an alternative to Microsoft Office and is widely used in education, public administration, and organizations that promote open technologies.

Components of LibreOffice

LibreOffice contains several applications, each designed for a specific type of task:

- **Writer:** Word processing (reports, letters, assignments)
- **Calc:** Spreadsheets (calculations, graphs, data analysis)



Chapter 2: Open Source Tools

2.3 LibreOffice Suite (Open-Source Office Suite)

LibreOffice is a powerful, free, and fully open-source office suite developed by The Document Foundation (TDF).

It serves as an alternative to Microsoft Office and is widely used in education, public administration, and organizations that promote open technologies.

Components of LibreOffice

LibreOffice contains several applications, each designed for a specific type of task:

- **Writer:** Word processing (reports, letters, assignments)
- **Calc:** Spreadsheets (calculations, graphs, data analysis)



Chapter 2: Open Source Tools

- **Impress:** Presentation design
- **Base:** Database creation and management

Features:

Free and open-source : no licensing fees.

Cross-platform support : available for Linux, Windows, and macOS.

Compatibility : can open and save documents in Microsoft Office formats (.docx, .xlsx, .pptx).

Extensions & templates : expand functionality for academic or professional use.

Collaborative editing (when used with Nextcloud or a document server).



Chapter 2: Open Source Tools

2.4 Collaboration Tools (File Sharing & Team Work)

Modern digital work environments rely heavily on tools that enable collaboration, communication, and secure file sharing.

Open-source ecosystems provide several powerful options.

2.4.1. Cloud Storage and File Sharing:

- a) **Nextcloud:** A self-hosted, open-source platform used for:
 - File storage
 - Online document editing
 - Calendar and task sharing
 - Team collaboration
- b) **OwnCloud:** Similar to Nextcloud but designed for enterprise-level collaboration.



Chapter 2: Open Source Tools

Nextcloud: A self-hosted, open-source platform used for:

- File storage

- Online document editing

- Calendar and task sharing

- Team collaboration

b) OwnCloud: Similar to Nextcloud but designed for enterprise-level collaboration.



Chapter 2: Open Source Tools

Benefits of Collaboration Tools:

Improved teamwork and communication

Easy access to files anywhere

Real-time collaboration

Enhanced productivity

Better version control and document safety



Chapter 2: Open Source Tools

2.5 Contributing to an Open-Source Project

Open-source contribution is a valuable learning and professional experience. It allows students to collaborate with global developers and improve software used worldwide.

Contributing means participating in the improvement of a project by:

Contributing to the Community:

- Answering questions on forums
- Sharing extensions or add-ons



Chapter 2: Open Source Tools

Writing documentation:

- Writing tutorials
- Reporting issues in the official documentation
- Proposing enhancements and improvements
- Translating the documentation

Bug Reporting:

- Testing new versions (including beta releases)
- Reporting issues by opening bug tickets
- Helping reproduce and understand problems
- Writing and submitting patches or fixes



Chapter 2: Open Source Tools

Features:

- Proposing new features
- Participating in discussions about feature design
- Implementing changes and new functionalities

Maintenance:

- Assisting with code review
- Helping integrate contributors' changes
- Supporting the release of new software versions

Distribution and Outreach:

- Promoting and raising awareness about the software
- Packaging the software for different distributions

Chapter 2: Open Source Tools

Getting Started: Good Practices for New Contributors:
Begin by understanding and following the project rules:
(both the written and unwritten ones).

- Read the essential documentation, including:
 - the license (LICENSE),
 - the README,
 - the contribution guide (CONTRIBUTING),
 - the code of conduct (CODE_OF_CONDUCT),
 - and any other reference files provided by the project

Chapter 2: Open Source Tools

- Familiarize yourself with the project's ecosystem by exploring:
 - the issue tracker and existing tickets,
 - community forums,
 - discussion channels (chat, mailing lists, etc.),
 - and by subscribing to project activity when possible.
- General Behavior Guidelines:
 - Be polite and respectful in all interactions.
 - Be patient — maintainers are volunteers; they owe contributors neither immediate attention nor answers.
 - Respond calmly to questions or feedback.
 - Always check the documentation before asking for help or opening an issue.

Chapter 2: Open Source Tools

How to contribute: a bug report?

If you discover a bug while using free software, you can report it to the software maintainers.

1. Look for the ticket system / bug tracker.
2. Search using different keywords to see if the bug has already been reported.

If it has, you can add any new information that helps reproduce or debug it.

3. If you haven't found a similar ticket, you can create a new one.

Make sure to include all useful information to reproduce the bug:

Chapter 2: Open Source Tools

Make sure to include all useful information to reproduce the bug:

- the software version,
 - how you obtained / compiled the package,
 - your environment (operating system, browser, etc.),
 - a precise sequence of steps / a file that allows the issue to be reproduced (as short as possible).
4. Remember to answer the maintainers' questions.